

INTRODUCTION À L'INTELLIGENCE ARTIFICIELLE

L'apprentissage est une modification d'un comportement sur la base d'une expérience.

Fabien Benureau¹

Dans ce cours², nous allons voir différents algorithmes qui cherchent à apprendre un comportement à partir d'exemples.

De tels algorithmes sont pertinents lorsque l'on fait face à un problème que l'on ne sait pas résoudre, mais pour lequel on dispose quand même d'une grande quantité d'exemples.

Vocabulaire : dans tout ce cours, on appelle « hyperparamètre » un paramètre qui permet de définir un problème, mais qui ne fait pas partie du contenu d'une instance (donc n'est pas donné en entrée aux algorithmes qui résolvent le problème). Par exemple, le 2 de 2-SAT est un hyperparamètre : il permet de définir le problème 2-SAT. Plus généralement, le k de k -SAT est un hyperparamètre : tous ces k -SAT sont des problèmes différents.

I - Généralités sur l'apprentissage supervisé

I.1 - Introduction

L'objectif de l'apprentissage supervisé est de trouver une façon d'associer le plus correctement possible des **observations** à des **étiquettes**. L'espace des étiquettes possibles est connu.

Définition 1

Dans toute cette partie, on note $\mathcal{X}$ l'ensemble des **observations** et $\mathcal{Y}$ celui des **étiquettes**.

On suppose également donné un **jeu d'entraînement** $\mathcal{D} = \{x_i, y_j\}$ de n observations-étiquettes, c'est à dire d'observations pour lesquelles on connaît la « bonne » étiquette.

Vocabulaire : On distingue les problèmes selon le nombre d'étiquettes possibles.

- S'il n'y a que deux étiquettes, on parle de **problèmes de classification binaire**. On dit que les étiquettes identifient une *classe*.

Exemples :

- Identifier si un mail (observation) est un spam ou non (étiquette).
- Identifier si une image contient une voiture.

- S'il y a une quantité finie >2 d'étiquettes, les étiquettes identifient des classes et on parle de **problèmes de classification multi-classes** :

Exemples :

- Identifier une fleur à partir de sa photo.
- Identifier la langue d'un texte.

- Si les étiquettes sont à valeurs dans $\mathbb{R}$, on parle de **régression**. Ex :

Exemples :

- Prédire le nombre de clics sur un lien.
- Prédire l'affinité de liaison entre deux molécules.

Définition 2

Apprentissage supervisé :

Exemple : On dispose de fruits qui ont une taille, un poids et une couleur (observations). Notre jeu d'entraînement contient les données étiquetées suivantes (les étiquettes sont les noms des fruits) :

1. Cette citation provient de *Introduction au Machine learning*, de Azencott. Très bon cours de L3 d'apprentissage si vous voulez aller plus loin.

2. Les splendides illustrations de ce cours sont issues du Azencott, du Balabonski et al., du cours de MPI de M.Bianquis, et du cours d'ITC de M.Larochette. Merci à elles et eux !

$$\mathcal{D} = \{ ((10\text{cm}, 400\text{g}, \text{jaune}), \text{banane}), \\ ((3\text{cm}, 10\text{g}, \text{rouge}), \text{fraise}), \\ ((50\text{cm}, 4\text{kg}, \text{orange}), \text{citrouille}), \\ ((45\text{cm}, 3.8\text{kg}, \text{orange}), \text{citrouille}), \\ ((7.5\text{cm}, 300\text{g}, \text{jaune}), \text{banane}) \}.$$

On pourrait attendre d'un apprentissage supervisé sur ce jeu d'entraînement qu'il fournisse, par exemple, la fonction

suivante : $f : \begin{cases} (_, _, \text{jaune}) \mapsto \text{banane} \\ (_, _, \text{rouge}) \mapsto \text{fraise} \\ (_, _, \text{orange}) \mapsto \text{citrouille} \end{cases}$

I.2 - Sur et sous-apprentissage

Définition 3

- On dit qu'un algorithme **sous-apprend** s'il est trop simple pour avoir de bonnes performances : il ne prend pas en compte assez d'éléments du jeu d'entraînement.
- On dit qu'un algorithme **sur-apprend** s'il est trop complexe et correspond « trop » au jeu d'entraînement, c'est à dire s'il capture aussi et avant tout le bruit de l'entraînement.

Exemple :

Sur la figure ci-dessous, on trace des lignes qui montre comment des modèles pourraient classifier des observations afin d'illustrer sous-apprentissage, sur-apprentissage et apprentissage correct.

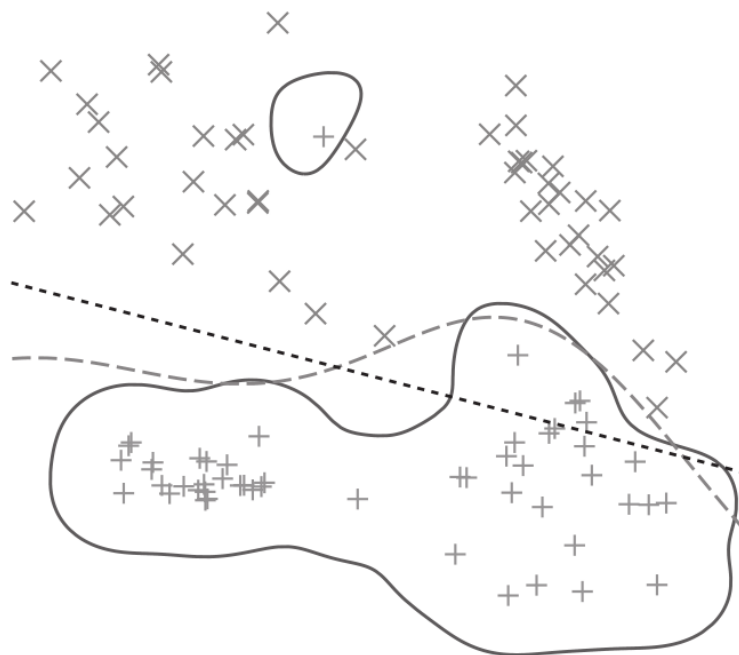


FIGURE 1 – En $\times$ et $+$ deux types d'observations. La ligne en pointillés longs apprend correctement. La ligne en trait continu sur-apprend : le $+$ en haut est probablement une erreur, et il n'y a pas de raison que les $+$ s'arrêtent en bas. La droite en petit traits sous-apprend : elle oublie toute une zone de $+$.

Pour évaluer les performances d'un algorithme, il faut le tester. Pour ce faire, une manière courante de procéder est la **validation croisée** :

- On sépare nos n données d'entraînement en m ensembles de tailles égales.
- On entraîne le modèle sur $m - 1$ ensembles, et on le teste sur le m -ème. On note son taux d'erreur.
- On répète cela m fois (pour chaque façon de choisir $m - 1$ parmi m).
- On en déduit un taux d'erreur « moyen ».

Remarques :

- $m = 5$ ou $m = 10$ sont des choix courants.

- Notez que l'évaluation des performances d'un algorithme demande d'avoir des données dont on connaît déjà l'étiquette car on veut pouvoir vérifier la réponse de l'algorithme. C'est généralement une mauvaise idée d'utiliser toutes les données de $\mathcal{D}$ pour entraîner, car on n'a alors plus de données pour tester et avoir une idée de la fiabilité du modèle.

I.3 - Matrice de confusion

Pour présenter de manière fine les performances d'un modèle, on peut procéder en utilisant une matrice de confusion :

Matrice de confusion :

On dit que l'on **normalise** la matrice de confusion lorsque :

Remarques :

- Une matrice de confusion est une représentation visuelle de la fiabilité d'un modèle.
- La somme des coefficients diagonaux de la matrice normalisée est la proportion de réponses correctes du modèle. Le taux d'erreur vaut $1 -$ cette proportion.

Exemple : Matrice de confusion (non normalisée) pour un apprentissage de reconnaissance de chiffres avec 3NN.

Dans cet exemple, le jeu d'entraînement $\mathcal{D}$ contient 5000 images de chiffres, et le jeu de test contient 1000 images (distinctes des données d'entraînement).

	0	1	2	3	4	5	6	7	8	9
0	84	0	0	0	0	0	1	0	0	0
1	0	126	0	0	0	0	0	0	0	0
2	2	4	98	0	1	0	2	7	2	0
3	0	2	0	95	0	2	2	2	2	2
4	0	1	0	0	98	0	1	2	0	8
5	0	1	0	2	0	78	2	0	2	2
6	2	0	0	0	1	0	84	0	0	0
7	0	5	1	0	1	1	0	90	0	1
8	3	1	0	3	2	3	2	0	71	4
9	0	0	0	0	2	0	0	2	1	89

FIGURE 2 – La première ligne et la première colonne indiquent quel chiffre (étiquette) est réel/prédit.

Exemples :

- Le coefficient $(2, 7)$ de la matrice contient le nombre d'images représentant le chiffre "2" pour lesquelles le modèle a prédit le chiffre "7".
- Combien de fois le modèle s'est-il trompé sur les images représentant le chiffre "5"?

On constate que le taux d'erreurs global est de 8.75%. Ce n'est pas mauvais pour si peu de données d'entraînement. Comment l'a-t-on calculé ?

Dans toute cette partie, on se donne $d : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}_+$ une fonction de distance sur les observations. d est :

- **Symétrique** : pour tous a, b de $\mathcal{X}$, $d(a, b) = d(b, a)$.
- **Définie Positive** : pour tous a, b de $\mathcal{X}$, $d(a, b) \geq 0$ et l'égalité est atteinte si et seulement si $a = b$.
- Et vérifie l'**inégalité triangulaire** : pour tous a, b, c de $\mathcal{X}$, $d(a, c) \leq d(a, b) + d(b, c)$.

Remarque :

- Le programme se concentre surtout sur une distance particulière : la distance euclidienne.

Idée naïve de classification :

On obtient ce que l'on appelle un **diagramme de Voronoï**. Par exemple, sur la figure ci-dessous, l'ensemble des observations possibles est le plan. Sont indiqués avec des points les observations du jeu d'entraînement. On a découpé le plan en cellules : chaque cellule est l'ensemble des points partageant la même plus proche observation d'entraînement.

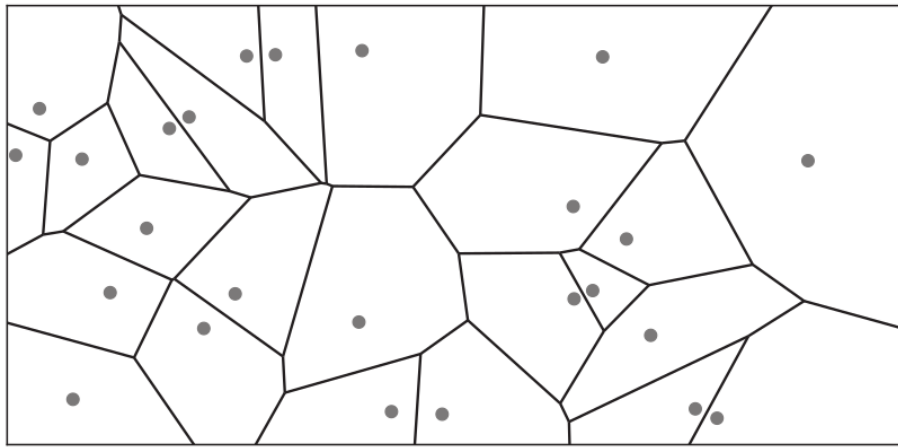


FIGURE 3 – Diagramme de Voronoï pour 25 points en deux dimensions. d est la distance euclidienne.

Cet algorithme de classification a un défaut : il suppose que le jeu d'entraînement est en quelque sorte « parfait », qu'il ne contient pas d'erreur. En pratique, il est commun que les observations soient « bruitées », qu'elles contiennent des approximations plus ou moins graves. On ne peut donc pas se fier à *une seule* association observation-étiquette du jeu d'entraînement.

Pour limiter la sensibilité à ces défauts de l'entraînement, on va choisir une étiquette non pas en fonction du plus proche voisin mais en fonction des k plus proches voisins³.

Soit $k \in \mathbb{N}^*$ un hyperparamètre.

La **méthode des k plus proches voisins** (« k nearest neighbors » en anglais, abrégé « kNN ») consiste à associer à une nouvelle observation x une étiquette dépendant des k plus proches observations de $\mathcal{D}$.

Formellement, en notant $\mathcal{N}_k(x)$ l'ensemble des k plus proches voisins de x , l'algorithme calcule la fonction f suivante :

3. Les noms explicites d'algorithmes sont toujours bienvenus.

- Pour un problème de classification :

$$f(x) = \arg \max_{y \in \mathcal{Y}} \sum_{i | x_i \in \mathcal{N}_k(x)} \mathbb{1}_{\{y_i = y\}}$$

- Pour un problème de régression :

$$f(x) = \frac{1}{k} \sum_{i | x_i \in \mathcal{N}_k(x)} y_i$$

Exemple : Quelques observations :

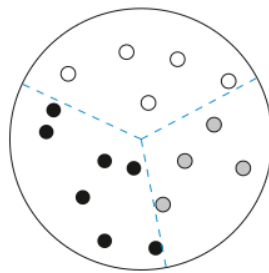


FIGURE 4 – Les observations sont en Blanc, Gris, Noir. On a ajouté la véritable frontière en pointillés.

Avec $k = 3$, voici comment seraient classifiées trois nouvelles observations a, b, c :

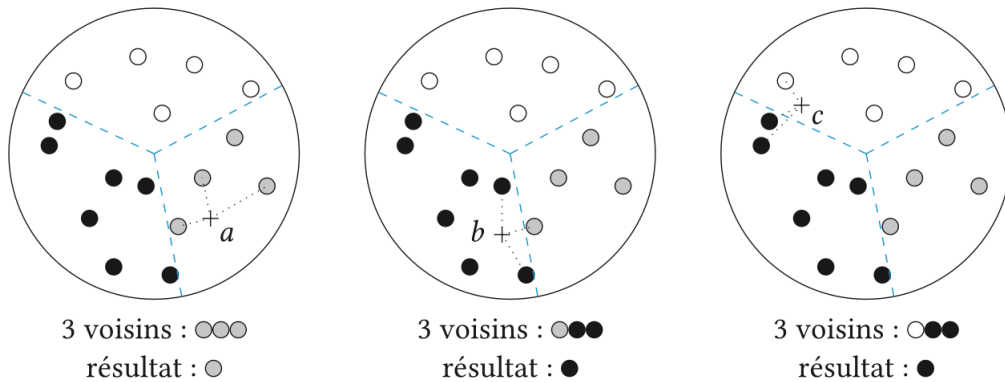


FIGURE 5 – 3NN : plus proches voisins et classification de a, b et c

On peut également regarder avec $k = 1$ ou $k = 5$:

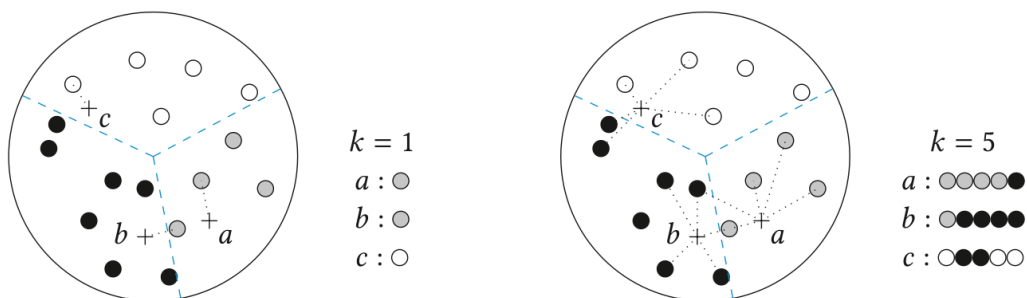


FIGURE 6 – 1NN et 5NN : plus proches voisins et classification de a, b et c

Remarques :

- D’autres exemples sont dans l’annexe en couleurs.
- A priori, il n’y a aucun entraînement particulier à faire : les observations étiquetées du jeu d’entraînement sont tout ce qui est nécessaire pour classer les nouvelles observations ensuite. En fait, comme nous le verrons plus tard, il peut être utile de structurer intelligemment $\mathcal{D}$, par exemple avec des arbres k dimensionnels.

Complexité naïve :

Étudions la complexité de la classification naïve d’une nouvelle observation x . On rappelle que $|\mathcal{D}| = n$, et on note t_d le temps de calcul d’une distance entre deux observations.

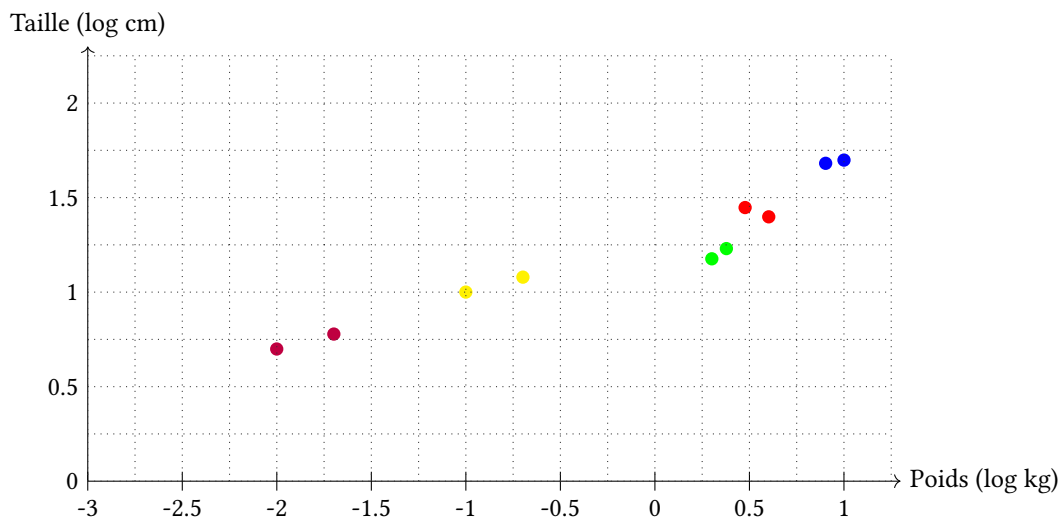
C’est plutôt lent. On dit parfois que k -NN est un algorithme *paresseux* : il a très peu d’entraînement (pour ne pas dire aucun), mais en contrepartie le coût de classification d’une valeur n’est pas négligeable⁴.

Exercice :

On souhaite développer un modèle qui reconnaît des animaux en fonctions de deux caractéristiques : leur poids en kilogramme, et leur taille en centimètres. On dispose du jeu d’entraînement $\mathcal{D}$ suivant :

$\mathcal{D} = \{ ((4 \text{ kg}, 25 \text{ cm}), \text{chat}), ((3 \text{ kg}, 28 \text{ cm}), \text{chat}),$
 $((10 \text{ kg}, 50 \text{ cm}), \text{chien}), ((8 \text{ kg}, 48 \text{ cm}), \text{chien}),$
 $((2 \text{ kg}, 15 \text{ cm}), \text{lapin}), ((2.5 \text{ kg}, 17 \text{ cm}), \text{lapin}),$
 $((0.1 \text{ kg}, 10 \text{ cm}), \text{oiseau}), ((0.2 \text{ kg}, 12 \text{ cm}), \text{oiseau}),$
 $((0.01 \text{ kg}, 5 \text{ cm}), \text{poisson}), ((0.02 \text{ kg}, 6 \text{ cm}), \text{poisson}) \}.$

Graphiquement ( les échelles sont logarithmiques) :



On souhaite maintenant classer les observations (animaux mystères) suivantes :

- Animal mystère 1 : (3 kg, 30 cm)
- Animal mystère 2 : (0,05 kg, 8 cm)

⁴. D’autant plus que si le jeu d’entraînement est très grand, il ne peut pas être conservé en RAM et doit donc être lu au fur et à mesure sur le disque dur à chaque fois, ce qui est bien plus lent.

En utilisant la méthode des k plus proches voisins pour $k = 3$, classez ces animaux mystères.

Réponses :

Choix de k

- $k = 1$ donne des diagrammes de Voronoï, qui ont l'inconvénient d'être trop sensibles aux erreurs dans le jeu d'entraînement $\mathcal{D}$.
- À l'inverse, $k = n$ revient à donner à toutes les observations la même étiquette, l'étiquette majoritaire dans $\mathcal{D}$, ce qui est généralement absurde.

Une première heuristique parfois utilisée est $k \simeq \sqrt{n}$. Mais il faut toujours rester critique vis-à-vis du comportement de notre modèle. On doit notamment se méfier des situations de sous-apprentissage et de sur-apprentissage.

Le mieux est de tester différentes valeurs de k et de les comparer (par validation croisée et/ou matrice de confusion).

II.2 - Accélérer kNN

Arbres d -dimensionnels

Idée générale :

Arbre d -dimensionnel :

Un **arbre d -dimensionnel** T pour un ensemble $S \subseteq \mathbb{R}^d$ est un arbre binaire dont les feuilles sont étiquetées par les éléments de S , et les noeuds internes sont étiquetés dans $\mathbb{N} \times S$ par l'indice d'une coordonnée et un élément de S .

L'ensemble des arbres d -dimensionnels pour S est défini par récurrence sur $|S|$ par :

Définition 7

Exemple :

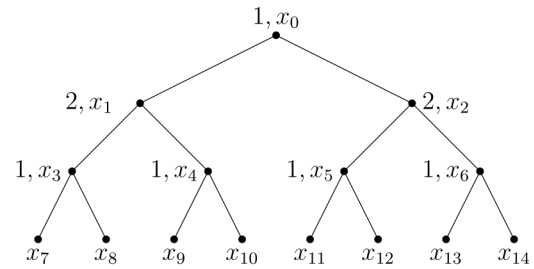
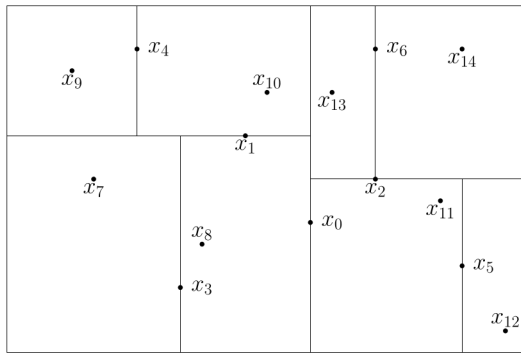


FIGURE 7 – Un arbre 2-dimensionnel pour représenter des points du plan.

Remarques :

- Généralement, on parle d'**arbre k -dimensionnel** (ou *k-dimensional tree*, abrégé *k-d tree*). Sauf que ce k n'a rien à voir avec le k des k plus proches voisins : l'un est une dimension d'un espace (le nombre de coordonnées des observations), l'autre un hyperparamètre (le nombre de plus proches voisins). C'est pourquoi dans ce cours on préfère parler d'arbre d -dimensionnel.
- Un noeud interne d'un arbre d -dimensionnel coupe S en S_- et S_+ . Par confort, dans la suite on dira que les sommets de S_- sont ceux qui sont « **à gauche** », et ceux de S_+ « **à droite** ». Bien sûr ces notions n'ont pas forcément de sens en dimension d , mais c'est plus pratique.
- Il existe une autre définition de ces arbres, où les noeuds internes ne sont pas étiquetés par des paires (dimension, point) mais par des paires (dimension, valeur). Cela permet certaines optimisations, mais complique un peu l'insertion.

Choix du couple (i, x) :

Il existe autant d'arbres d -dimensionnels possibles pour un même ensemble S que de choix de couples (i, x) à chaque étape. Pour avoir un arbre efficace, le choix de la bonne dimension i et du bon point x n'est pas simple :

- **Arbre équilibré** : Une première façon de choisir est de chercher à équilibrer l'arbre. Cela donne un arbre optimal pour la recherche d'appartenance d'un point à S . Pour équilibrer l'arbre, il faut s'assurer qu'il coupe en deux l'espace des points, autrement dit il faut que S_- et S_+ soient de même taille.

Algorithme possible de construction de l'arbre équilibré :

- **Recherche de voisins rapide** : Une autre façon de faire est de ne pas chercher à faire un arbre équilibré, mais plutôt un arbre où la recherche des plus proches voisins est la plus rapide. En effet, comme nous allons le voir, rechercher le plus proche voisin demande de considérer plusieurs « cellules » de l'espace séparé (celles où peut se trouver le plus proche voisin). Il est alors intéressant d'essayer de diminuer le nombre maximal de « cellules » que l'on sera amené à explorer (i.e. diminuer le nombre de sous-branches de l'arbre que l'on explore, quitte à ce qu'elles soient plus longues).

Algorithme de recherche des k plus proches voisins

En toute généralité, la recherche des k plus proches voisins à l'aide d'un arbre utilise la notion d'**hyperplan**⁵. Dans ce cours, on a simplement besoin de savoir que dans $\mathbb{R}^d$, si on note i une coordonnée et v une valeur, alors $\{x \in \mathbb{R}^d | x_i = v\}$ (i.e. « on fixe la coordonnée i à v ») est un hyperplan (techniquement, un **hyperplan affine**; on se contentera de dire hyperplan).

Pour simplifier, tous les exemples seront faits en 2D. Autrement dit, dans la suite, vous pouvez remplacer « hyperplan » par « droite obtenue en fixant la valeur d'une coordonnée ».

Mini-exercice :

5. Vous verrez les hyperplans en mathématiques, si ce n'est pas déjà fait.

Dans le plan, quel est l'hyperplan défini par « abscisse = 2 » ? Par « ordonnée = -1 » ?

On considère un ensemble S de n points de $\mathbb{R}^d$, un arbre d -dimensionnel T qui les contient, et un point y de $\mathbb{R}^d$ dont on veut trouver les plus proches voisins dans S .

Idée générale de l'algorithme :

RECHERCHE DES k PLUS PROCHES VOISINS (à l'aide d'un arbre d -dimensionnel) :

Entrées : $S \subseteq \mathbb{R}^d$, $y \in \mathbb{R}^d$ et T un arbre d -dimensionnel pour S .

Initialisation : On pose $\mathcal{V} = \emptyset$. Cet ensemble contiendra tout au long de l'algorithme les k plus proches voisins de y vus jusqu'à maintenant.

Cas de base :

Si l'arbre T est réduit à une feuille étiquetée $s \in S$, **on met à jour $\mathcal{V}$ avec s** . Une mise à jour consiste à faire les opérations suivantes :

- Si $\mathcal{V}$ contient moins de k éléments, on ajoute s à $\mathcal{V}$.
- Sinon, on compare s avec l'élément p_0 de $\mathcal{V}$ le plus éloigné de y . Si s est plus proche de y que p_0 , on enlève p_0 de $\mathcal{V}$ et on lui ajoute s , sinon on ne modifie rien.

Cas inductif :

Notons (i, x) l'étiquette de la racine de T (qui est un noeud interne). Notons $\mathcal{H}_i$ l'hyperplan des points dont la i -ème coordonnée est égale à x_i . On distingue deux cas, selon la position de y par rapport à $\mathcal{H}_i$.

Si $y_i > x_i$:

- **On explore récursivement « à droite »**, i.e. dans S_+ (qui contient y). À la fin de cette exploration, $\mathcal{V}$ contient donc les k plus proches voisins de y dans S_+ .
- **On met à jour $\mathcal{V}$ avec le point x** .
- On explore ensuite récursivement la branche de gauche, mais uniquement si c'est nécessaire. C'est automatiquement le cas si on n'a pas encore trouvé k voisins ($\mathcal{V}$ contient moins de k éléments).

Sinon, notons p_0 le point le plus éloigné de y parmi les k plus proches voisins actuels $\mathcal{V}$, et notons r sa distance à y . On la compare avec la distance c de y à $\mathcal{H}_i$. **Voir le schéma ci-dessous.**

- Si cette distance c est plus grande que r , alors il est **inutile d'explorer « à gauche »**, dans S_- . En effet, tous les éléments de S_- sont « à gauche » de l'hyperplan $\mathcal{H}_i$, donc éloigné de y d'une distance au moins c , supérieure à celle du pire voisin p_0 déjà trouvé. C'est dans ce cas qu'on élague l'arbre.
- Sinon, il peut y avoir un plus proche voisin de l'autre côté de l'hyperplan. **On explore récursivement « à gauche »**.

Si $y_i \leq x_i$: On procède de même de manière symétrique.

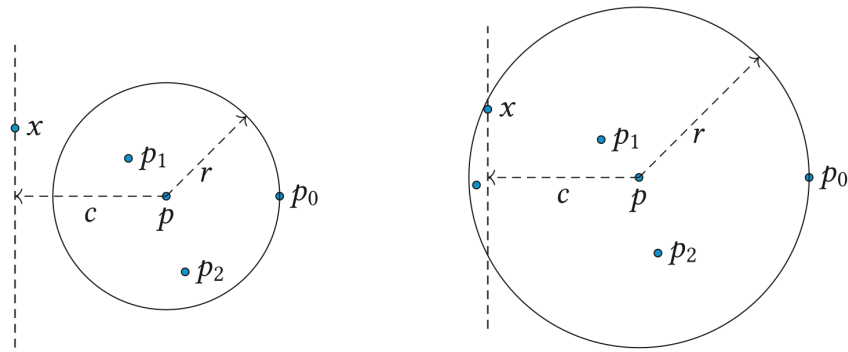


FIGURE 8 – Ici, on recherche les 3 plus proches voisins de p . L'hyperplan $\mathcal{H}_i$ est la droite en pointillé (abscisse de x). p_0 , p_1 et p_2 sont les trois plus proches voisins trouvés « à droite ».

Exemple :

Appliquons cet algorithme (à la main) pour trouver les 3 plus proches voisins d'un point y ajouté à la figure 7 à l'emplacement suivant (voir tableau) :

(À faire sur feuille !!)

Remarques :

- Pour des petites valeurs de k , la structure de données la plus efficace pour $\mathcal{V}$ est un tableau trié. Sur des grandes valeurs, c'est une file de priorité dont la priorité est l'éloignement à l'élément cherché (ainsi l'élément le plus facilement accessible de la file de priorité est le voisin le plus éloigné de y).
- Si la version d -dimensionnelle vous paraît compliquée, concentrez-vous sur le fonctionnement en deux dimensions.
- On voit sur l'exemple précédent que pour que la recherche des k plus proches voisins soit efficace, le tout n'est pas d'avoir un arbre équilibré : il faut un arbre qui garantisse le plus d'élagages possibles, autrement dit que l'on ait le moins souvent possible à aller explorer les deux enfants d'un même noeud. Plusieurs heuristiques hors-programme existent ; la page Wikipédia anglaise des *k-d tree* en parle bien.

III - Algorithme ID3

III.1 - Introduction

Dans toute cette partie, on va étiqueter les observations en utilisant un **arbre de décision** déterminé à partir des données d'entraînement.

Exemple illustratif : le jeu du « Qui est-ce ? »

À faire sur feuille !!

Remarques : ⚠ Cet exemple illustratif possède quelques défauts :

- Cet exemple est uniquement là pour présenter les arbres de décisions, pas l'apprentissage supervisé ! Ces données ne ressemblent pas du tout à un vrai jeu d'entraînement ou à de véritables observations à classer par IA. Ici, on n'a qu'une seule observation (personnage) par étiquette (nom), et cette étiquette n'a aucun rapport a priori avec les critères observés (il n'y a pas beaucoup de rapport entre le nom et l'apparence physique d'une personne). Ça n'aurait pas de sens d'apprendre dessus.

Nous reviendrons sur cet exemple pour comprendre la notion d'entropie.

- Dans le cas du « Qui est-ce ? », le "meilleur arbre" est l'arbre équilibré. Ceci est dû au fait que chaque classe est équilibrée (une personne par nom).

⚠ Ce n'est pas forcément le cas ! Comme dans le cas de l'**algorithme de Huffman**, on peut vouloir construire un arbre déséquilibré pour des effectifs déséquilibrés, afin de minimiser le nombre de questions à poser en moyenne pour étiqueter une observation. Par exemple, si 90% des observations devraient être étiquetées "chien", et seulement 1% "perroquet", on veut pouvoir vite aboutir à la conclusion "chien", quitte à devoir poser beaucoup de questions pour étiqueter un "perroquet".

L'algorithme ID3 fait partie d'une catégorie d'algorithmes qui cherchent à construire, à partir du jeu d'entraînement, un arbre de décision pour l'étiquetage des observations, qui soit le plus fiable et rapide possible. L'apprentissage (i.e. la construction de l'arbre) doit trouver une suite de critères permettant de bien classer les observations.

Cadre formel :

On note $c_0, c_1, \dots, c_{m-1}$ les étiquettes. On travaille en dimension d , et on se limite dans le cadre du programme à des **caractéristiques binaires**. Ainsi, nos observations sont des éléments de $\{\text{Vrai}; \text{Faux}\}^d$.

Remarque :

- Comme on construit un arbre de décision binaire qui teste chaque caractéristique au plus une fois, il sera de profondeur au plus d . Les feuilles sont les étiquettes.

On veut déterminer quels attributs tester, et dans quel ordre. On cherche à privilégier les attributs qui font gagner le plus d'information, c'est à dire qui minimisent l'entropie ⁶

III.2 - Entropie de Shannon

Définition

L'entropie est une quantité qui permet de quantifier la quantité d'information dont on dispose. Intuitivement, ici, elle représente le « **nombre moyen de questions binaires** auxquelles il faut répondre pour déduire l'étiquette ». Définissons la plus proprement :

6. En physique, l'entropie d'un système est une mesure « de son désordre », ou encore « le nombre d'états microscopiques dans lequel il peut réellement être en ne connaissant que des mesures macroscopiques ». Autrement dit, c'est mesurer un nombre de « possibilités », et ce nombre de possibilités diminue si l'on gagne de nouvelles informations.

Entropie :**Remarques :**

- Interprétation : $H(E)$ est l'**incertitude sur l'étiquette** que l'on a en tirant un élément uniformément au hasard dans E . On admet que cette quantité permet bien de mesurer cela⁷.
La base du logarithme ($\log_2$) est très importante si l'on veut pouvoir aussi utiliser l'interprétation en « nombre de questions binaires ».
- On a $p_0 + p_1 + \dots + p_{m-1} = 1$, les étiquettes (réelles) partitionnent l'ensemble des observations.
- La valeur minimale possible pour l'entropie est $H(E) = 0$: tous les $\log_2 p_i$ sont négatifs ($0 \leq p_i \leq 1$), et le cas d'égalité a lieu lorsque tous les p_i valent 0 ou 1, c'est à dire que toutes les observations partagent la même étiquette. On a alors une information parfaite, il n'y a besoin d'« aucune question » pour déterminer l'étiquette d'une observation tirée au hasard, donc l'entropie est minimale (et même nulle).
- Valeur maximale possible : $H(E) = \log(m)$, si les observations sont uniformément réparties entre chaque étiquette. C'est dans ce cas qu'on dispose du moins d'information, une observation a autant de chance d'être de n'importe quelle étiquette, et l'entropie est maximale. Plus il y a de classes (étiquettes) possibles, plus l'entropie est élevée.
- (HP) En toute généralité, l'entropie est définie pour une variable aléatoire X quelconque à valeurs dans $\{c_0, \dots, c_{m-1}\}$, plutôt que pour un ensemble d'observations. On la définit comme :

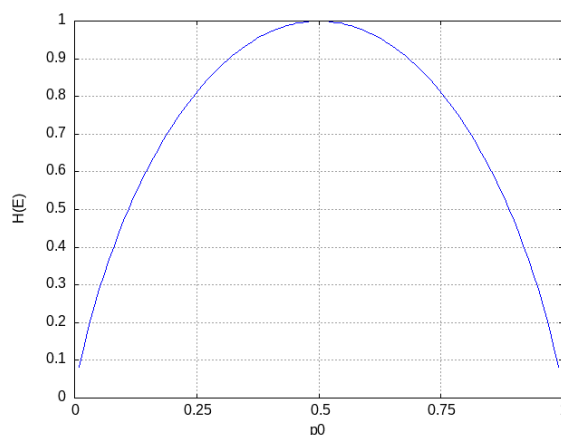
$$\mathbb{E}(-\log_2(\mathbb{P}(X))) = - \sum_{i=0}^{m-1} \mathbb{P}(X = c_i) \log_2(\mathbb{P}(X = c_i)),$$
d'où les noms p_i plutôt que « f_i » (en toute généralité, ce sont des probabilités). Notez que la probabilité $\mathbb{P}(X)$ est elle-même une variable aléatoire.

Exemple :

Pour $p_0 = 5\%$, $p_1 = 1\%$, $p_2 = 17\%$, $p_3 = 38\%$, $p_4 = 8\%$ et $p_5 = 31\%$, on obtient $H(E) = 2.063$.

Parenthèse : l'entropie binaire

On parle d'entropie **binnaire** lorsqu'on se limite à deux étiquettes c_0 et c_1 . Dans ce cas, on peut obtenir une visualisation utile pour comprendre intuitivement le comportement de H . En exploitant le fait que $p_0 + p_1 = 1$, on trace le graphe de l'entropie binaire $H(E)$ en fonction de la valeur de p_0 :

FIGURE 9 – $H(E)$ en fonction de p_0

7. Vous approfondirez cette notion pendant vos études supérieures en cours de théorie de l'information, si vous poursuivez dans ce domaine. Si cela vous dérange, dites vous que cette formule d'entropie est une heuristique qui fait marcher l'apprentissage d'arbres de décisions.

Observations :

- L'entropie est **maximale** quand $p_0 = p_1 = 0.5$. En effet, c'est dans cette situation que l'on sait le moins dans quel état se trouve un élément de E : il peut tout aussi bien être c_0 que c_1 .
- Par contre, si $p_0 = 0.75$, un élément quelconque a plus de chances d'être étiqueté c_0 : on connaît avec meilleure probabilité son étiquette (elle vaut probablement c_0), donc l'entropie est moindre.
- Le cas extrême $H(E) = 0$ correspond aux cas où tous les éléments de E ont la même étiquette (soit c_0 soit c_1). Dans ce cas, on a une **information parfaite** : on sait parfaitement quelle est leur étiquette, il n'y a aucun doute, d'où l'entropie nulle.

À retenir : l'entropie correspond à l'incertitude que l'on a sur l'étiquette d'un élément.

Gain d'information

Gain d'information :

Définition 9

Remarques :

- Interprétation : Pour parler à votre intuition, le gain d'information correspond en quelque sorte à « la nouvelle information moins l'ancienne », ou encore « l'ancienne incertitude (entropie) moins la nouvelle ». La nouvelle entropie sachant l'information $x_i = V$ dépend des entropies des deux ensembles où $x_i = V$ et où $x_i = F$.

Exemple :

Illustrons cette formule de gain d'information sur un schéma-exemple (et notamment la présence des fractions). Supposons E constitué de n observations, tel que :

- $x \in E$ vérifie $x_i = V$
- et pour tout $y \in E \setminus \{x\}$, $y_i = F$.

Si toutes les autres coordonnées sont uniformément réparties entre Vrai et Faux dans $E \setminus \{x\}$, on obtient la situation suivante :

L'un des ensembles ($E_{x_i=F}$) a une entropie maximale $H = \log_2(n-1)$, et l'autre ensemble ($E_{x_i=V}$) a une entropie nulle. Sans autre information, on pourrait donc se dire que globalement, on a gagné beaucoup d'information (l'un des deux ensembles est maintenant parfaitement connu). Mais en moyenne on n'a presque pas gagné d'information, car cela concerne un unique élément. C'est un peu comme si je demande "Est ce que c'est Jacques?" en première question au « Qui est-ce ? ».

On comprend alors que l'on ne considère pas à égalité les entropies des ensembles $E_{x_i=V}$ et $E_{x_i=F}$. Il faut les pondérer par leurs effectifs respectifs, d'où la formule obtenue.

- Si l'on ne se limite plus à des arbres binaires, c'est à dire à $\{\text{Vrai}; \text{Faux}\}^d$, alors il ne faut pas uniquement considérer $E_{x_i=V}$ et $E_{x_i=F}$ mais tous les $E_{x_i=\dots}$ pour toutes les valeurs possibles de la dimension i : on comprend qu'un tel arbre risquerait de vite exploser en taille⁸.
- (HP) Derrière cette formule se cache en réalité et en toute généralité une espérance conditionnelle (et une entropie conditionnelle...).

8. Mais entre nous, la question en IA n'est pas de savoir si un modèle explose en taille, mais plutôt à quel point il explose en taille. En effet, pour être bon un modèle doit d'une façon ou d'une autre stocker de l'information, beaucoup d'information : cela a toujours un coût.

Exemple concret de gain d'information :

Pour simplifier la visualisation, faisons un exemple légèrement plus général. Au lieu de considérer des observations dans $\{\text{Vrai}; \text{Faux}\}^d$, on considère des observations dans $\mathbb{R}^d$ (ici, dans le plan), pour lesquelles on regarde certaines propriétés qui s'évaluent à Vrai ou Faux.

Considérons les données suivantes, qui peuvent avoir trois étiquettes (Blanc, Gris, Noir) :

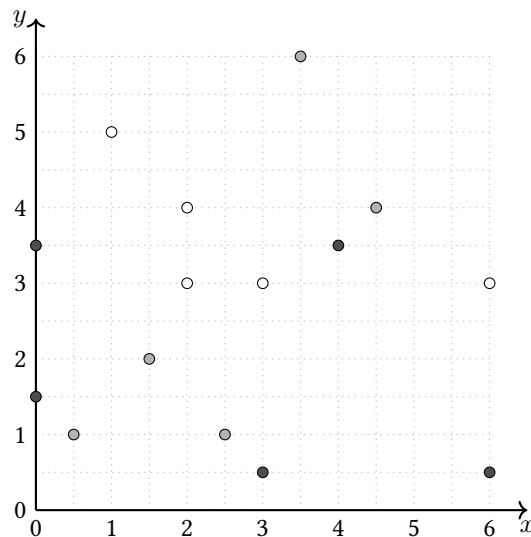


FIGURE 10 – Des données E avec trois étiquettes possibles.

Dans cet exemple, il y a 15 données. Il y en a autant de Blanches, de Grises et de Noires :

$$p_B = p_G = p_N = \frac{1}{3}.$$

Entropie :

On obtient : $H(E) =$

Il faut donc « 1.58 questions binaires » en moyenne pour décider de l'étiquette d'un élément choisi au hasard dans cet ensemble.

Gain d'information :

Considérons maintenant la propriété « $y \geq 3$ » : elle scinde nos données en deux sous-ensembles :

$$E_- = \{(x, y) \in E \mid y < 3\} \quad \text{et} \quad E_+ = \{(x, y) \in E \mid y \geq 3\}.$$

On peut calculer :

– Dans E_- , il y a 6 points. On a $p_B = 0, p_G = \frac{1}{2}, p_N = \frac{1}{2}$. Donc $H(E_-) =$

– Dans E_+ , il y a 9 points. On a $p_B = \frac{5}{9}, p_G = \frac{2}{9}, p_N = \frac{2}{9}$. Donc $H(E_+) =$

On peut maintenant calculer le gain d'information obtenu par le fait de savoir si $y \geq 3$ ou non :

$$G(E, \text{«} y \geq 3 \text{»}) =$$

Autrement dit, savoir si $y \geq 3$ ou non fait gagner en moyenne $\frac{1}{3}$ d'une question binaire pour décider de l'étiquette, sachant qu'il en fallait en moyenne 1.58 au début. Notez que ce n'est donc pas une question très efficace en moyenne (on espère qu'une question binaire nous fasse gagner... une question binaire.)

III.3 - L'Algorithme ID3

Présentons maintenant l'algorithme ID3, pour *Iterative Dichotomizer 3*.

Idée générale :

ALGORITHME ID3 :

Entrées : Un jeu d'entraînement $\mathcal{D}$, c'est à dire un ensemble E d'observations dont on connaît les étiquettes.

Cas de base : On distingue trois cas de base différents :

- **Si tous les éléments de E partagent une même étiquette**, on crée une unique feuille correspondant à cette étiquette.
- **Si E est vide**, on crée une feuille étiquetée par l'étiquette majoritaire parmi les observations couvertes par le noeud parent.

Ce cas est utile notamment pour les cas où l'on cherchera à étiqueter une observation dont les caractéristiques sont très différentes de celles de toutes les observations du jeu d'entraînement. On se retrouve alors dans notre arbre de décision à emprunter un chemin qui ne correspond à aucune de nos données d'entraînement.

- **Si on est à profondeur d dans l'arbre construit**, alors on a déjà testé toutes les dimensions (chacune des coordonnées $x_i = V/F$). Dans ce cas, on crée une feuille étiquetée par l'étiquette majoritaire parmi les observations de E .

Ce cas est utile notamment si nos critères ne sont pas assez exhaustifs pour départager entièrement des observations ayant des étiquettes différentes, mais possédant les mêmes caractéristiques examinées par le modèle.

Cas inductif :

On choisit l'attribut i qui maximise le gain d'information, $i = \arg \max_{j \in [0; d-1]} G(E, j)$, et on crée un noeud étiqueté par cet attribut. On **construit récursivement** le sous-arbre gauche correspondant à $E_{x_i=F}$ (et on étiquette la branche qui y mène par « Faux »), et le sous-arbre droit correspond à $E_{x_i=F}$ (et on étiquette la branche qui y mène par « Vrai »).

Remarque :

- Si on est dans le troisième cas de base et qu'il y a plusieurs étiquettes différentes dans E , alors la connaissance des d coordonnées ne suffit pas à connaître l'étiquette. Dans ce cas, il faut soit s'en contenter (si les performances sont malgré tout bonnes), soit changer d'observations (pour prendre en compte plus de paramètres). Changer d'algorithme n'aidera pas puisque l'arbre contient déjà toute l'information que l'on peut avoir.

Exemple :

Considérons des livres d'informatique qui peuvent ou non avoir certaines propriétés. Ces propriétés sont de savoir s'il y a des exercices (E), des corrections (C), s'ils sont écrits en français (F), et s'ils utilisent de vrais langages de programmation (L). On appellera S l'ensemble des observations (E étant utilisé pour un des attributs) :

Les étiquettes sont ici également binaires : les lecteurs recommandent-ils cet ouvrage ou non.

Voici les données d'entraînement (il y a 12 livres/observations) :

E	C	F	L	recomm.
oui	oui	oui	oui	oui
oui	oui	oui	non	non
oui	oui	non	oui	oui
oui	oui	non	non	non
oui	non	oui	oui	oui
oui	non	oui	non	non
oui	non	non	oui	non
oui	non	non	non	non
non	non	oui	oui	non
non	non	oui	non	non
non	non	non	oui	oui
non	non	non	non	non

FIGURE 11 – Jeu d'entraînement de recommandation de livres

On peut calculer les gains d'information (écrire les formules en fonction des entropies) :

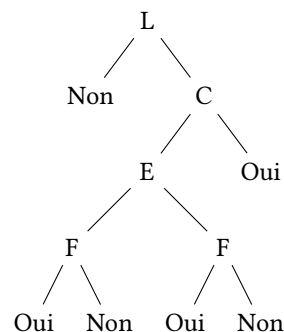
$$G(S, E) =$$

$$G(S, C) =$$

$$G(S, F) =$$

$$G(S, L) =$$

On en déduit que l'attribut faisant gagner le plus d'information est L : il sera donc à la racine de l'arbre. On continue ensuite à gauche et à droite. En répétant le processus, on obtient :



Remarques :

- ⚠ Selon les branches, les attributs ne seront pas forcément testés dans le même ordre.
- Il y a un vrai risque de sur-apprentissage : certaines des coordonnées des observations peuvent ne pas être pertinentes pour ce que l'on cherche à apprendre. Imaginez par exemple que l'on veuille apprendre à reconnaître un ornythorinque, une poule et un lézard, et que nos observations soient des triplets (présence de poils, présence de bec(s), âge du capitaine Hadock). L'âge du capitaine est clairement un critère inutile, mais qui peut créer un sur-apprentissage.
- ID3 fournit un arbre qu'un humain peut facilement comprendre. De manière générale, un gros inconvénient des modèles d'apprentissage par arbre est que l'on comprend leur fonctionnement, que le résultat n'est pas une simple boîte noire.
Toutefois, dès que l'on sort d'espaces binaires, les arbres de ID3 sont soit trop simplistes et sous-apprennent, soit trop gros. Il faut alors utiliser des variantes plus fines.

III.4 - Au-delà de ID3 (HP)

D'autres critères et optimisations pour faire de bons arbres existent :

- Ne pas aller jusqu'à la profondeur d mais trouver un bon critère d'arrêt qui prévienne le sur-apprentissage et/ou l'explosion de l'arbre.
- Construire plusieurs arbres distincts à partir d'un même jeu d'entraînement, et utiliser la réponse majoritaire parmi ces arbres pour répondre à une requête.
- L'algorithme **Adaboost** pousse l'idée précédente un cran plus loin en apprenant à quels arbres on peut ou non faire confiance sur des requêtes typiques. Autrement dit, il construit un vote pondéré entre les arbres pour déterminer la classe. Cet algorithme et ses développements successifs sont assez efficaces et marquants pour avoir valu à leurs auteurs le prix Gödel.

IV - Apprentissage non-supervisé

IV.1 - Problèmes des k moyennes

Jusqu'à présent, nos algorithmes apprenaient sur un jeu de données d'entraînement déjà étiqueté. On s'intéresse dans cette partie à un autre problème : le problème de *clustering* non-supervisé.

Oublions ici la notion de classe. Le but du clustering n'est pas d'étiqueter des observations (même si on peut l'utiliser pour cela), c'est de faire des paquets les plus resserrés possible. Plus formellement, on cherche à résoudre **le problème des moyennes**⁹.

Le problème des moyennes, appelé k -MOYENNES, est le problème d'optimisation suivant :

Instance :

Solution :

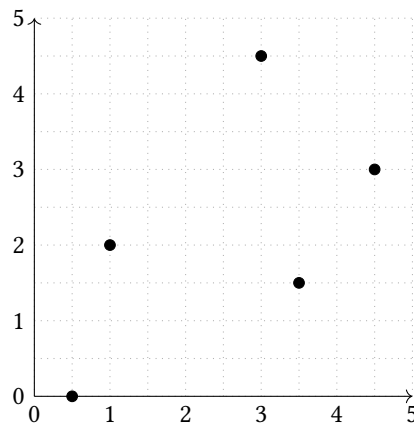
Mesure à minimiser :ⁱ

Les ensembles $S_0, S_1, \dots, S_{k-1}$ de la partition sont souvent appelés **clusters** (mot issu du terme anglais).

i. L'isobarycentre d'une famille de points est la moyenne de ces points, i.e. le (nouveau) point dont les coordonnées sont les moyennes des coordonnées correspondantes des points.

Exemple :

1. Calculer l'isobarycentre de ces points :



2. Répartissez ces points en 2 clusters de sorte à résoudre le problème des moyennes avec $k = 2$.
Et en 3 clusters ?

IV.2 - Difficulté du problème

Le problème de décision associé au problème des moyennes est NP-complet.

Exercice :

1. Définir le problème de décision associé au problème des moyennes.

9. Le vrai nom est « k -moyennes » : il est malheureux, car k n'est pas un hyperparamètre. Retenez plutôt « problème des moyennes ».

2. Montrer que le problème de décision associé au problème des moyennes est dans NP :

3. Proposer un algorithme pour résoudre le problème des k -moyennes (en temps exponentiel)¹⁰ :

Remarques :

- Une façon de le prouver NP-dur est de réduire une variante de NOTALLEQUAL-3SAT, abrégé NAE-3SAT, au problème des moyennes. NAE-3SAT est une restriction de 3SAT où l'on impose que les trois valeurs au sein d'une clause ne doivent pas toutes être égales.
- Le problème reste NP-dur y compris lorsque l'on fixe la valeur de k , pour tout $k \geq 2$. Dans ce cas, on fait de k un hyperparamètre. On est dans la même situation que CNF-SAT (k est une entrée) vs les k -SAT où SAT est un hyperparamètre.
- Pour une distance euclidienne, il existe une 9-approximation en temps polynomial pour le problème des moyennes.

IV.3 - Algorithme de Loyd

L'algorithme de Loyd est une méthode de résolution du problème des moyennes, par apprentissage non-supervisé.

10. Si vous en trouvez un en temps polynomial, appelez-moi !

Idée générale :

ALGORITHME DE LOYD :

Entrées : Un ensemble $E \subseteq \mathbb{R}^d$ d'observations.

Initialisation :

Choisir k observations **au hasard dans** E , et les nommer « centres ».

Boucle :

- **Partitionner E en k clusters** en associant chaque point de E au centre le plus proche. (Les observations associées à un même centre forment un cluster.)
- Pour chaque cluster, **remplacer le centre par l'isobarycentre des points du cluster**. (Les centres ne sont alors plus nécessairement des observations.)

Répéter ces deux étapes tant qu'elles modifient les clusters construits.

Résultat :

Lorsque les deux étapes de la boucle ne modifient plus les clusters, on dit que l'algorithme **a convergé**. On renvoie alors les clusters obtenus.

Exemple : Cf annexe couleurs

Remarques :

- Un cluster associé à un centre n'est jamais vide (invariant).
- Il n'y a aucune garantie d'optimalité. C'est en quelque sorte un optimum local : modifier légèrement les centres ne modifie plus les clusters en fin d'algorithme ; mais il n'y a rien qui garantisse une optimalité absolue.
- Une implémentation naïve donne *chaque* itération en $O(nkd)$ car pour chaque point (il y en a n au total), on calcule sa distance à chacun des k centres en temps $O(d)$ (d coordonnées) et on en prend le minimum. Dans le pire des cas, l'algorithme peut itérer un nombre exponentiel de fois en n avant de converger : le pire des cas peut-être très long. En pratique, le nombre d'itérations est raisonnable.
- Les centres initiaux sont aléatoires : lancer l'algorithme plusieurs fois peut donner des solutions de qualité diverses. Une façon d'améliorer l'algorithme est de le lancer plusieurs fois et garder le meilleur *clustering* obtenu. Une variante fournissant de très bons résultats (**KMEANS++**) consiste à initier les centres comme suit : le premier est une observation choisie au hasard uniformément. Ensuite, prendre une autre observation au hasard biaisé : la probabilité d'être choisi est proportionnelle au carré de la distance aux centres déjà choisis. Répéter jusqu'à avoir tous les centres initiaux, puis utiliser l'algorithme de Loyd usuel.
- L'algorithme de Loyd est parfois appelé « **algorithme des k -moyennes** ». Insistons : k est un paramètre, une entrée de la fonction et non un hyperparamètre ¹¹ !!

11. Vraiment, le nom « k -moyennes » n'est pas un cadeau...

IV.4 - Algorithme CHA

Un(e) autre (famille d') algorithme résolvant ce problème est la Classification Hiérarchique Ascendante (CHA). Voici son schéma général :

Algorithme 1 : Classification Hiérarchique Ascendante

Entrées : Un ensemble de N observations $x_1, \dots, x_N$, non étiquetées.

1
2
3
4
5
6
7
8

La condition « Tant que nécessaire » peut comprendre plusieurs **conditions d'arrêts** différentes :

- S'arrêter lorsqu'on atteint un nombre cible de clusters.
- S'arrêter lorsque les clusters restants sont suffisamment éloignés les uns des autres. (Dans ce cas, l'éloignement cible doit être supérieur à la plus petite distance entre deux observations, ou on bouclera indéfiniment.)
- D'autres critères existent.

Il faut aussi décider quelle distance sur les clusters on considère, pour pouvoir déterminer les deux clusters « les plus proches » :

Il existe plusieurs façons de définir la **distance** $\delta(C, C')$ **entre deux clusters** C et C' .

Soit $d(x, y)$ une distance sur les observations. Les fonctions suivantes définissent des distances valides entre clusters :

- **Liaison simple** : $\delta(C, C') = \min_{x \in C, x' \in C'} d(x, x')$
- **Liaison complète** : $\delta(C, C') = \max_{x \in C, x' \in C'} d(x, x')$
- **Liaison barycentrique** : $\delta(C, C') = d(\mu_C, \mu_{C'})$ avec $\mu_C, \mu_{C'}$ les isobarycentres de C et C' .
- Et d'autres encore

Schéma :

Exemple :

Appliquez l'algorithme CHA sur les points ci-dessous. On utilisera la liaison complète. On s'arrêtera lorsqu'il ne restera plus que 3 clusters :

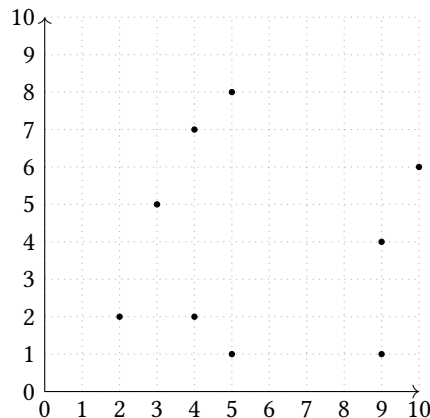


FIGURE 12 – Déroulé (à compléter) de l'algorithme de Loyd

Remarques :

- Cet algorithme est un algorithme glouton.
- Il existe une implémentation de cet algorithme dont le temps d'exécution est en $O(n^3)$. Pour l'atteindre, on a besoin de mémoriser les distances $\delta(\dots, \dots)$ pour ne pas toutes les recalculer à chaque fois. La complexité spatiale est alors en $\Omega(n^2)$. Ne pas mémoriser les distances permet d'économiser de l'espace, mais la complexité temporelle empirerait alors d'une puissance de n . On comprend que cet algorithme est peu utilisé sur de gros jeux de données : il demande trop d'espace et de temps.

V - Fléau de la dimension

L'expression « fléau de la dimension » regroupe de nombreux phénomènes qui font que les performances des algorithmes d'apprentissage chutent rapidement lorsque la dimension de l'espace des observations augmente. Par exemple :

- Quand d augmente, il faut exponentiellement plus de données d'entraînement pour avoir des performances comparables.
- Les arbres d -dimensionnels ne sont plus efficaces lorsque d est grand, car la probabilité que l'on doive aller « de l'autre côté de l'hyperplan » est bien plus élevée (admis).
- Les arbres de ID3 ont une taille exponentielle en d .

VI - Annexes en couleurs : de jolies illustrations

VI.1 - À propos de k -NN

On veut reconnaître différentes iris à l'aide de k -NN. Pour cela, on considère deux¹² paramètres pour les fleurs : la longueur des pétales, et la longueur des sépales.

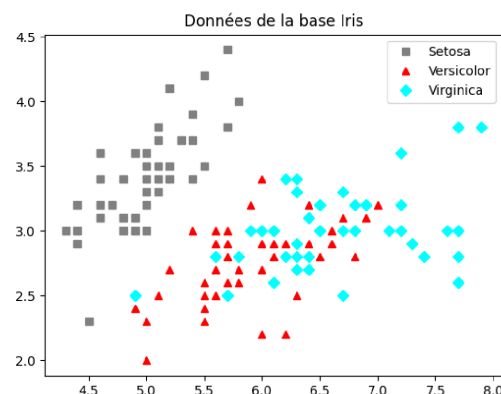


FIGURE 13 – Le jeu d'entraînement des Iris.

12. Pratique pour représenter dans le plan;)

Vous remarquerez que ce jeu d'entraînement est moins « propre » que celui du disque : les zones intuitives semblent se chevaucher, la frontière n'est pas nette. Les exemples réels de jeu d'entraînement sont en effet souvent imparfaits sur cet aspect... et encore, ce jeu d'entraînement est sans erreur, avec uniquement deux dimensions et trois classes.

Voici les zones de classification obtenues pour différentes valeurs de k :

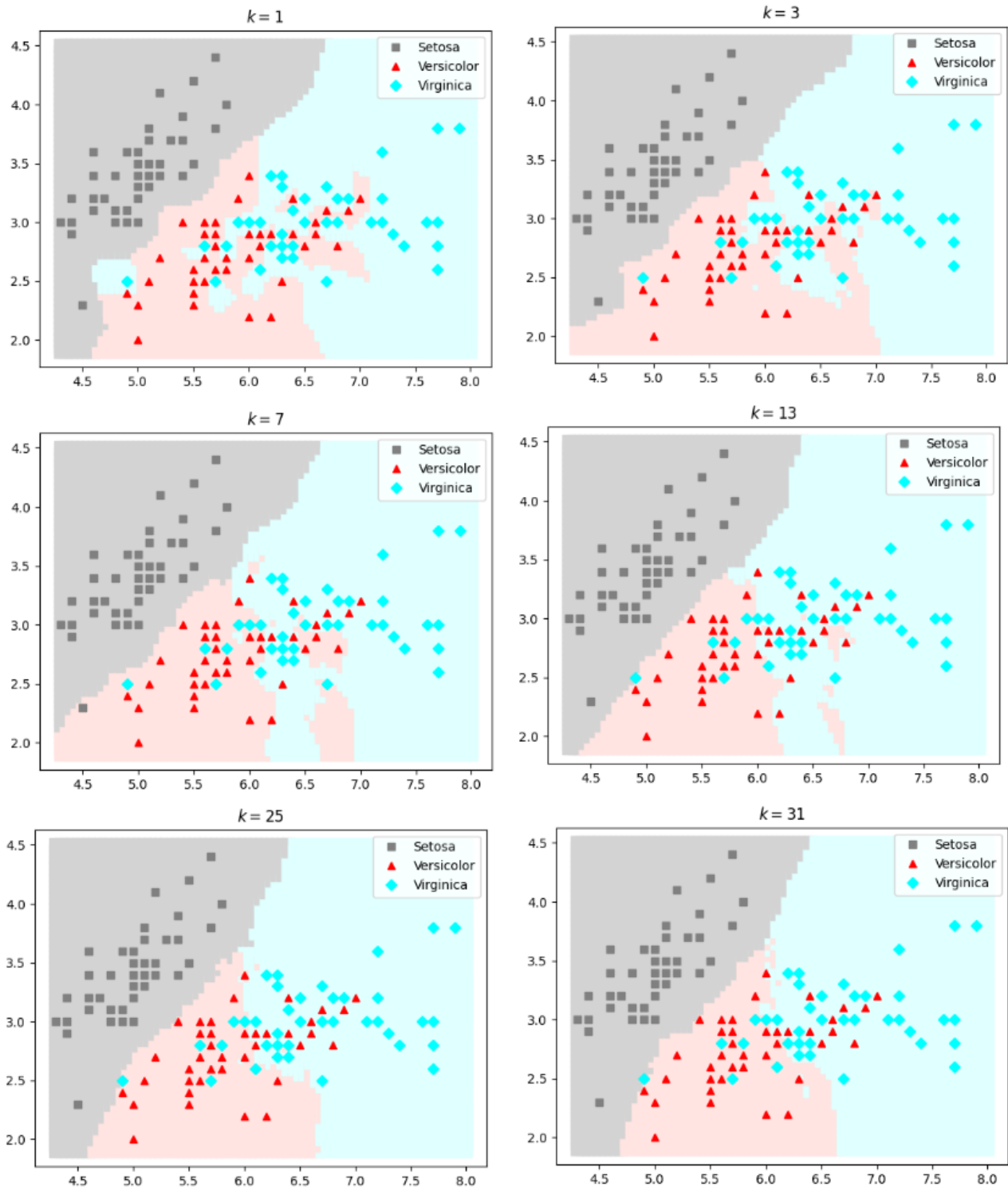


FIGURE 14 – Zones de classification par k -NN pour différentes valeurs de k .

En apprentissage, on parle de **frontière de décision** pour désigner les éléments qui pourraient être classés d'une façon ou d'une autre. Elle se comprend bien sur ces schémas.

VII - Apprentissage non-supervisé

VII.1 - Autour de l'algorithme de Lloyd

Voici un exemple où l'algorithme de Lloyd converge en 4 étapes. Les données sont coloriées selon leur cluster¹³ Les lignes de séparation des clusters (aka les frontières des Voronoï associées aux isobarycentres) sont indiquées ; ignorez les points oranges dessus.

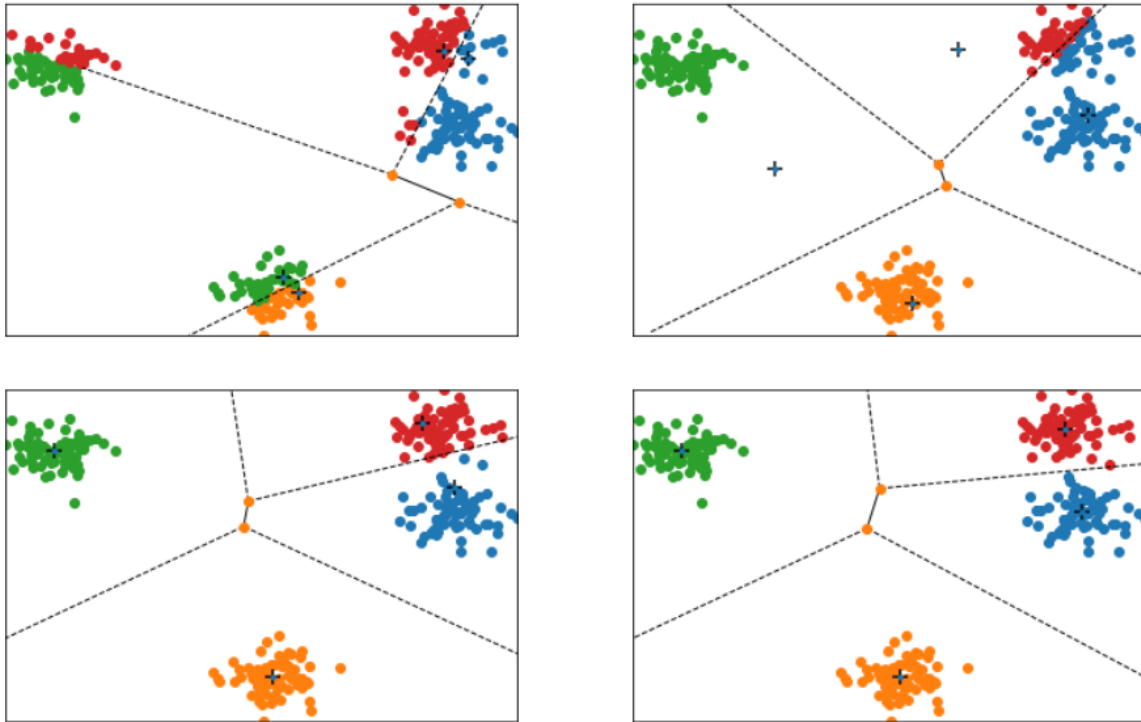


FIGURE 15 – Un déroulé de l'algorithme de Lloyd qui converge en 4 étapes.

13. Si vous vous demandez pourquoi ne pas les colorier par étiquette : elles n'ont pas d'étiquette ! C'est de l'apprentissage non-supervisé.